The Critical Line Algorithm

A parametric active-set method for the entire mean–variance efficient frontier, as implemented in `cvxcla`

Thomas Schmelzer
Jebel Quant Research, Abu Dhabi

June 16, 2026

Abstract

We describe the Critical Line Algorithm (CLA) of Markowitz as implemented in the `cvxcla` package. The CLA computes the *entire* mean–variance efficient frontier of a long-only (or box-constrained) portfolio problem exactly, rather than sampling it point by point. It exploits the fact that the efficient frontier is piecewise linear in the weight space: between two consecutive *turning points* the optimal weights are an affine function $w(\lambda) = \alpha + \lambda\beta$ of a single scalar parameter λ that trades expected return against variance. The algorithm starts at the maximum-return portfolio ($\lambda = \infty$) and walks the frontier down to the minimum-variance portfolio ($\lambda = 0$), changing the status of exactly one asset at each turning point. Each step amounts to a single solve of a reduced Karush–Kuhn–Tucker (KKT) system, handled here by block elimination through a small Schur complement. We document the problem formulation, the optimality conditions, the event logic that detects turning points, the anti-cycling rule used on degenerate problems, and the covariance-operator abstraction that lets structured risk models (diagonal plus low rank) avoid ever materialising a dense $n \times n$ matrix.

Contents

1 Introduction **2**

2 Problem formulation **3**
2.1 The parametric quadratic program . . . 3
2.2 Optimality conditions . . . 3
2.3 Piecewise-linearity: the affine segment . . . 3

3 The first turning point **4**

4 The turning-point iteration **4**
4.1 Solving the reduced KKT system by block elimination . . . 4
4.2 Reduced gradients for blocked assets . . . 5
4.3 Event detection . . . 5
4.4 Anti-cycling on degenerate problems . . . 6
4.5 Termination and the minimum-variance endpoint . . . 6
4.6 The complete procedure . . . 6

5 The covariance operator abstraction **7**
5.1 Dense covariance . . . 7
5.2 Incremental dense covariance . . . 7
5.3 Factor (diagonal-plus-low-rank) covariance . . . 8

6 The efficient frontier object **8**

7 Complexity and properties **8**

8 Relation to the Bailey–López de Prado algorithm **9**

9 Numerical experiment **10**
9.1 Illustration: a 20-asset frontier . . . 10
9.2 Runtime versus problem size . . . 11
9.3 What the runtime gap is, and is not . . . 12
9.4 Runtime versus factor rank . . . 13
9.5 Exactness against a reference QP solver . . . 13
9.6 Empirical example: the S&P 500 frontier . . . 14

10 Conclusion **14**

Acknowledgements **15**

1 Introduction

The mean–variance portfolio selection problem of Markowitz (1952) asks for the allocation of capital across n assets that minimises risk for a given level of expected return. Sweeping the return level produces a one-parameter family of optimal portfolios whose risk–return image is the *efficient frontier*. The Critical Line Algorithm (CLA), introduced by Markowitz (1956) and developed in Markowitz (1959), computes this frontier *exactly and in full*: it returns a finite list of *turning points* between which the frontier is described in closed form, so no discretisation or repeated re-optimisation is required. It belongs to a broad family of parametric quadratic-programming and active-set methods for the efficient frontier (Wolfe, 1959; Perold, 1984; Niedermayer and Niedermayer, 2010; Bailey and López de Prado, 2013); Markowitz and Todd (2000) give the canonical description and a reference implementation, while Perold (1984), Stein et al. (2008) and Hirschberger et al. (2010) develop large-scale and active-set variants.

We claim no new turning points — the frontier is Markowitz's, and `cvxcla` traces exactly the same one — but a new *interface* to it: we capture the covariance contract as three operations (a matrix–vector product, a free-block solve, and a free-to-blocked cross product) so that structured risk models trace the identical frontier without touching the algorithm. The one element here that is, to our knowledge, not standard in existing CLA implementations — which operate on an explicit dense covariance (Markowitz and Todd, 2000; Bailey and López de Prado, 2013) — is this *covariance-operator abstraction*. The turning-point loop touches the covariance through only three operations: a matrix–vector product, a solve against the free-asset block, and a free-to-blocked cross product. Capturing exactly this contract as a small protocol decouples the algorithm from the covariance representation, so a structured risk model — e.g. a diagonal-plus-low-rank factor covariance solved through the Woodbury identity — drops in as a backend and traces the *same*

exact frontier at a fraction of the cost, without a single change to the turning-point loop. This is the paper's main engineering contribution; the experiments of §9 quantify the resulting speed-up.

This document describes the implementation in the `cvxcla` package. The core of the implementation is the class `CLA` in `src/cvxcla/cla.py`; it is supported by `first.py` (the starting portfolio), `operators.py` (the covariance backend), and `types.py` (the turning-point and frontier data structures). The notation below follows the code: μ is the vector of expected returns, Σ the covariance, A, b the linear equality constraints, and ℓ, u the box bounds on the weights.

2 Problem formulation

2.1 The parametric quadratic program

Let $w \in \mathbb{R}^n$ be the vector of portfolio weights, $\mu \in \mathbb{R}^n$ the expected returns, $\Sigma \in \mathbb{R}^{n \times n}$ the (symmetric, positive semidefinite) covariance matrix, $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ the equality constraints, and $\ell, u \in \mathbb{R}^n$ the lower and upper bounds. The implementation traces the frontier through the *return-parametrised* quadratic program

$$\begin{aligned} \min_{w \in \mathbb{R}^n} \quad & \tfrac{1}{2}\, w^{\mathsf{T}} \Sigma\, w \;-\; \lambda\, \mu^{\mathsf{T}} w \\ \text{subject to} \quad & A w = b, \\ & \ell \le w \le u, \end{aligned} \tag{1}$$

where $\lambda \ge 0$ is a scalar that weights expected return against variance. The canonical instance has a single equality constraint, the budget constraint $\mathbf{1}^{\mathsf{T}} w = 1$ (so $A = \mathbf{1}^{\mathsf{T}}$ and $b = 1$), but (1) and the implementation allow any m linear equalities.

As λ decreases from $+\infty$ to 0, the optimiser of (1) moves from the maximum expected-return portfolio (subject to the constraints) to the global minimum-variance portfolio. The locus of these optimisers is exactly the efficient frontier.

2.2 Optimality conditions

At a fixed λ the program (1) is convex, so the Karush–Kuhn–Tucker (KKT) conditions are necessary and sufficient. Partition the assets into a *free* set $\mathcal{F}$ (weights strictly inside their bounds) and a *blocked* set $\mathcal{B} = \mathcal{F}^c$ (weights pinned at a bound, $w_i \in \{\ell_i, u_i\}$). For the free assets the stationarity condition holds with no bound multiplier:

$$\Sigma_{\mathcal{F}\mathcal{F}}\, w_{\mathcal{F}} + \Sigma_{\mathcal{F}\mathcal{B}}\, w_{\mathcal{B}} + A_{\mathcal{F}}^{\mathsf{T}} \nu \;=\; \lambda\, \mu_{\mathcal{F}}, \qquad A_{\mathcal{F}}\, w_{\mathcal{F}} + A_{\mathcal{B}}\, w_{\mathcal{B}} \;=\; b, \tag{2}$$

where $\nu \in \mathbb{R}^m$ is the multiplier of the equality constraints and the subscripts select the free/blocked rows and columns. The blocked weights $w_{\mathcal{B}}$ are known (they sit on ℓ or u), so (2) is a linear system in the unknowns $(w_{\mathcal{F}}, \nu)$.

2.3 Piecewise-linearity: the affine segment

The crucial structural fact is that, *while the free/blocked partition is fixed*, the right-hand side of (2) is affine in λ and the system matrix is constant. Hence the solution is affine in λ:

$$w(\lambda) \;=\; \alpha \;+\; \lambda\, \beta, \tag{3}$$

where $\alpha, \beta \in \mathbb{R}^n$ are constant on the segment (the blocked entries of β are zero; the blocked entries of α equal the active bound values). The implementation calls these vectors `r_alpha` and `r_beta`. Equation (3) is the *critical line*: a straight line in weight space valid for an interval of λ. A *turning point* is a value of λ at which the partition $(\mathcal{F}, \mathcal{B})$ must change, i.e. an endpoint of such an interval.

3 The first turning point

The walk starts at $\lambda = +\infty$, where variance is irrelevant and (1) degenerates into "earn the most expected return that the bounds and budget allow." This portfolio is computed in closed form by `init_algo` (`src/cvxcla/first.py`):

1. Initialise every weight at its lower bound, $w \leftarrow \ell$.
2. Sort the assets by descending expected return.
3. Walk down that order, raising each asset's weight from ℓ_i towards u_i, accumulating capital, until the budget $\sum_i w_i = 1$ is reached (within a floating-point tolerance). The last asset touched is only *partially* filled — it absorbs the residual $1 - \sum_i w_i$ — and is the single *free* asset of the first turning point. All others are blocked, either at their lower bound (not yet reached) or upper bound (fully filled).

The result is "the smallest subset of assets with highest return whose upper bounds sum to at least one," which is the maximum-return vertex of the feasible polytope. It is returned as a `TurningPoint` with $\lambda = \infty$, a weight vector, and a boolean `free` mask. The tolerance $1 - 10^{-12}$ on the budget test matters: the increment $1 - \sum_i w_i$ can only reach 1 up to rounding, and without the slack the loop would skip past the genuinely interior asset and mark the next (zero-weight) asset as free.

4 The turning-point iteration

From the first turning point the algorithm repeatedly: (i) solves the reduced KKT system for the current partition to obtain the affine segment (3); (ii) finds the largest λ below the current one at which an *event* occurs; (iii) updates the partition by the single asset responsible for that event; and (iv) records the new turning point. It stops when λ reaches 0.

4.1 Solving the reduced KKT system by block elimination

For the free partition $\mathcal{F}$ the system (2) is the saddle-point (KKT) system

$$\begin{bmatrix} \Sigma_{\mathcal{F}\mathcal{F}} & A_{\mathcal{F}}^{\mathsf{T}} \\ A_{\mathcal{F}} & 0 \end{bmatrix} \begin{bmatrix} w_{\mathcal{F}} \\ \nu \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix}. \tag{4}$$

Because the right-hand side splits into a constant part and a λ-part, the implementation solves (4) for *two* right-hand sides simultaneously (the "α system" and the "β system"):

$$r_1^{\alpha} = -\Sigma_{\mathcal{F}\mathcal{B}}\, w_{\mathcal{B}}, \quad r_2^{\alpha} = b - A_{\mathcal{B}}\, w_{\mathcal{B}}; \qquad r_1^{\beta} = \mu_{\mathcal{F}}, \quad r_2^{\beta} = 0.$$

Rather than assembling and factoring the full saddle matrix, the code eliminates the free block first (*block elimination* / Schur complement). With

$$Y = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} A_{\mathcal{F}}^{\mathsf{T}}, \qquad z^{\alpha} = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} r_1^{\alpha}, \qquad z^{\beta} = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} r_1^{\beta},$$

obtained from a *single* multi-column solve against $\Sigma_{\mathcal{F}\mathcal{F}}$, the equality multipliers come from the $m \times m$ Schur complement $S = A_{\mathcal{F}}\, \Sigma_{\mathcal{F}\mathcal{F}}^{-1} A_{\mathcal{F}}^{\mathsf{T}} = A_{\mathcal{F}} Y$:

$$\nu^{\alpha} = S^{-1}(A_{\mathcal{F}} z^{\alpha} - r_2^{\alpha}), \qquad \nu^{\beta} = S^{-1}\left(A_{\mathcal{F}} z^{\beta}\right). \tag{5}$$

Back-substitution then gives the free part of the affine segment,

$$\alpha_{\mathcal{F}} = z^{\alpha} - Y\nu^{\alpha}, \qquad \beta_{\mathcal{F}} = z^{\beta} - Y\nu^{\beta}, \tag{6}$$

while the blocked entries are fixed: $\alpha_{\mathcal{B}} = w_{\mathcal{B}}$ and $\beta_{\mathcal{B}} = 0$. The dominant cost is the multi-right-hand-side solve against the free block $\Sigma_{\mathcal{F}\mathcal{F}}$; the Schur solve (5) is only $m \times m$, and for the standard budget constraint $m = 1$, so it is a scalar division.

4.2 Reduced gradients for blocked assets

To decide when a blocked asset should re-enter the free set we need the reduced gradient of the Lagrangian at the blocked coordinates. The implementation forms two vectors, again split into a constant and a λ-linear part:

$$\gamma = \Sigma\,\alpha + A^{\mathsf{T}}\nu^{\alpha}, \tag{7}$$

$$\delta = \Sigma\,\beta + A^{\mathsf{T}}\nu^{\beta} - \mu. \tag{8}$$

For a blocked asset i the quantity $\gamma_i + \lambda\,\delta_i$ is (proportional to) its bound multiplier along the segment. While that multiplier keeps the correct sign the asset is correctly blocked; the λ at which it would change sign is exactly the value at which the asset wants to become free. (The covariance products $\Sigma\alpha$ and $\Sigma\beta$ are applied through the covariance operator of §5, never as an explicit dense multiply when a structured backend is in use.)

4.3 Event detection

Walking down from the current λ, the segment (3) stays optimal until one of two things happens. The implementation enumerates four event *types*, two for each direction:

A free asset reaches a bound (free → blocked). A free weight evolves as $w_i(\lambda) = \alpha_i + \lambda\beta_i$. It meets a bound at

$$\lambda_i \;=\; \begin{cases} \dfrac{u_i - \alpha_i}{\beta_i}, & \beta_i < 0 \quad \text{(heads to the } \textit{upper} \text{ bound)}, \\[2ex] \dfrac{\ell_i - \alpha_i}{\beta_i}, & \beta_i > 0 \quad \text{(heads to the } \textit{lower} \text{ bound)}. \end{cases} \tag{9}$$

A blocked asset's multiplier changes sign (blocked → free). A blocked asset wants to become free at

$$\lambda_i \;=\; -\frac{\gamma_i}{\delta_i}, \tag{10}$$

admissible only when the asset is at its *upper* bound with $\delta_i < 0$, or at its *lower* bound with $\delta_i > 0$.

These candidate ratios are assembled into a matrix $L \in \mathbb{R}^{n\times 4}$ (one column per event type), with entries set to $-\infty$ where the event cannot occur. Two numerical guards apply:

- **Slope floor.** Slopes β_i and derivatives δ_i are tested against $\sqrt{\varepsilon_{\text{mach}}}$, not the user tolerance. A free weight with a *tiny but nonzero* slope still crosses a bound given a long enough λ range, so filtering at the coarser tolerance would let weights drift out of bounds; only slopes at floating-point noise level (below $\sqrt{\varepsilon_{\text{mach}}}$) are discarded, since the enormous ratios they would produce merely amplify rounding error.

- λ **window.** The segment is valid only for λ at or below the current value, because the frontier is traced with non-increasing λ. Candidate ratios above the current λ (by more than the tolerance) are spurious crossings outside the segment and are set to $-\infty$.

The next turning point is at $\lambda^\star = \max_{i,t} L_{it}$. If $\lambda^\star < 0$ the frontier is complete.

4.4 Anti-cycling on degenerate problems

On degenerate problems — tied expected returns, duplicated assets, or several constraints active at once — multiple events can occur at the same ratio $\lambda^\star$. A naive "pick any maximiser" rule can cycle (the same partitions recurring without progress). The implementation uses a **Bland-style rule**: among all events within the tolerance of $\lambda^\star$ it selects the one with the lowest asset index, and within an asset the lowest event-type column. This makes the choice deterministic and resolves degeneracies one asset per iteration.

Termination follows by the standard anti-cycling argument. There are only finitely many free/blocked partitions (at most 2^n), and each partition fixes the affine segment (3) and hence the interval of λ over which it is optimal. The parameter λ is non-increasing along the walk, so the only way to fail to terminate is to revisit a partition at the same $\lambda^\star$ — a cycle. Bland's lowest-index selection rule rules this out exactly as it does for the simplex method: among the events tied at $\lambda^\star$ the deterministic lowest-index choice cannot generate a cyclic sequence of partitions. With cycles excluded and the partition set finite, the walk visits each partition at most once and terminates in finitely many steps.

The asset and event type selected determine the partition update: event types "free $\to$ blocked" clear the asset's `free` flag, while "blocked $\to$ free" set it. Exactly one asset changes status. The new weight vector $w(\lambda^\star) = \alpha + \lambda^\star \beta$ is stored as the next turning point.

4.5 Termination and the minimum-variance endpoint

The loop runs while $\lambda > 0$. When no admissible event has a positive ratio the walk has reached the global minimum-variance portfolio; the algorithm appends a final turning point at $\lambda = 0$ with weights α (the constant part of the current segment). Termination is guaranteed in exact arithmetic by the anti-cycling argument of §4.4; the hard iteration cap of $100(n+1)$ is a *numerical* safety net rather than the guarantee itself. A correct trace empirically runs $O(n)$ times, so far exceeding this cap signals a floating-point failure (e.g. a tolerance-induced cycle), which is raised loudly rather than allowed to hang. Every stored turning point is validated to satisfy the bounds and the budget constraint before being accepted.

4.6 The complete procedure

Algorithm 1 collects the steps.

Algorithm 1. Critical Line Algorithm (as implemented in `cvxcla`).

Input: mean μ, covariance Σ, bounds ℓ, u, equality constraints A, b, tolerance τ.

1. Compute $w^{(0)} \leftarrow$ `init_algo`(μ, ℓ, u), the maximum-return vertex (§3); append it with $\lambda \leftarrow \infty$.
2. **While** $\lambda > 0$:
   (a) Read the free/blocked partition $(\mathcal{F}, \mathcal{B})$ of the last turning point; classify $\mathcal{B}$ into *at-upper* / *at-lower* and fix $w_{\mathcal{B}}$ to the active bounds.
   (b) Solve the free block $\Sigma_{\mathcal{F}\mathcal{F}}$ for Y, z^{α}, z^{β} (multi-RHS, §4.1).
   (c) Form the Schur complement $S = A_{\mathcal{F}} Y$ and solve (5) for $\nu^{\alpha}, \nu^{\beta}$; back-substitute (6) to get α, β.
   (d) Compute the reduced gradients γ, δ via (7)–(8).
   (e) Assemble the candidate ratios L from (9)–(10); apply the slope floor and the λ window. Set $\lambda^{\star} \leftarrow \max L$; **if** $\lambda^{\star} < 0$ **break**.
   (f) Pick the event by the Bland-style rule (§4.4) and flip the `free` status of that one asset. Set $\lambda \leftarrow \lambda^{\star}$ and append the turning point $w \leftarrow \alpha + \lambda^{\star} \beta$.
3. Append the final turning point $w \leftarrow \alpha$ at $\lambda = 0$ (the minimum-variance portfolio) and **return** the list of turning points.

5 The covariance operator abstraction

The turning-point loop touches the covariance matrix through only three operations: a full matrix–vector product Σx (`matvec`); a solve against the free block, $\Sigma_{\mathcal{F}\mathcal{F}}^{-1} R$ for a multi-column right-hand side R (`solve_free`); and a free-to-blocked cross product $\Sigma_{\mathcal{F}\mathcal{B}} x_{\mathcal{B}}$ (`cross`). The package captures exactly this contract in the `CovarianceOperator` protocol (`src/cvxcla/operators.py`), so the algorithm never assumes an explicit matrix. Three backends implement it.

5.1 Dense covariance

`DenseCovariance` wraps an explicit symmetric $n \times n$ matrix and implements the three operations with plain `numpy` calls: `matvec` is Σx, `solve_free` is `np.linalg.solve` on the principal submatrix $\Sigma_{\mathcal{F}\mathcal{F}}$, and `cross` is the off-diagonal block product. This reproduces exactly the behaviour of the classical CLA on a raw matrix and is the reference backend.

5.2 Incremental dense covariance

`IncrementalDenseCovariance` wraps the same explicit matrix and returns identical results, but computes `solve_free` differently. The reference backend factorises $\Sigma_{\mathcal{F}\mathcal{F}}$ from scratch at every turning point, an $O(n_{\mathcal{F}}^3)$ cost. Because the CLA changes the free set by exactly one asset between consecutive turning points (§4.4), this backend instead *maintains* the explicit inverse $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}$ and updates it in place as the free set changes. When an asset enters the free set, a rank-one bordered update appends its row and column through the Schur scalar $s = \Sigma_{aa} - c^{\mathsf{T}} \Sigma_{\mathcal{F}\mathcal{F}}^{-1} c$ (with $c = \Sigma_{\mathcal{F}a}$ the new cross-column); when an asset leaves, the symmetric deletion formula $\Sigma_{\mathcal{F}'\mathcal{F}'}^{-1} = B_{\setminus p} - B_{\cdot p} B_{p\cdot} / B_{pp}$ drops its row and column from the current inverse B. Each update costs $O(n_{\mathcal{F}}^2)$ rather than $O(n_{\mathcal{F}}^3)$,

so the linear-algebra strategy matches the incremental-inverse baseline of §9.3: on a dense problem of a few hundred assets it runs roughly twice as fast per solve, and traces the identical frontier to machine precision.

The trade is numerical. A maintained inverse accumulates floating-point error over the $O(n)$ rank-one updates of a trace, whereas the fresh solve is clean at every step; a non-positive or non-finite Schur pivot, or any free-set change that is not a single-asset flip, triggers a full refactorisation as a guard. For well-conditioned, positive-definite problems the two backends agree to solver precision, but on ill-conditioned or near-degenerate inputs the plain `DenseCovariance` — or the factor backend below — is the safer choice. This backend is therefore offered as an opt-in fast path, not the default, and the win is dense-only: the factor backend never forms $\Sigma_{\mathcal{F}\mathcal{F}}$ at all, so there is no inverse to maintain.

5.3 Factor (diagonal-plus-low-rank) covariance

`FactorCovariance` represents a structured risk model

$$\Sigma \;=\; \operatorname{diag}(d) \;+\; U\,\Delta\,U^{\mathsf{T}}, \qquad d \in \mathbb{R}^{n}_{>0},\; U \in \mathbb{R}^{n\times k},\; \Delta \in \mathbb{R}^{k\times k}, \tag{11}$$

the form taken by factor models (Sharpe, 1963) and by random-matrix-theory-cleaned covariances (constant-residual-eigenvalue / eigenvalue clipping). The free-block solve never forms $\Sigma_{\mathcal{F}\mathcal{F}}$; instead it uses the Woodbury identity

$$\Sigma_{\mathcal{F}\mathcal{F}}^{-1} \;=\; D_{\mathcal{F}}^{-1} \;-\; D_{\mathcal{F}}^{-1} U_{\mathcal{F}}\, W^{-1}\, U_{\mathcal{F}}^{\mathsf{T}} D_{\mathcal{F}}^{-1}, \qquad W \;=\; \Delta^{-1} + U_{\mathcal{F}}^{\mathsf{T}} D_{\mathcal{F}}^{-1} U_{\mathcal{F}} \;\in\; \mathbb{R}^{k\times k}, \tag{12}$$

so a solve costs $O(n_{\mathcal{F}}k^2 + k^3)$ rather than $O(n_{\mathcal{F}}^3)$, and no $n \times n$ matrix is ever allocated — memory and per-solve cost scale as $O(nk)$ instead of $O(n^2)$. The cross product (11) contributes only through its low-rank part, since the diagonal has no off-diagonal block: $\Sigma_{\mathcal{F}\mathcal{B}}\, x_{\mathcal{B}} = U_{\mathcal{F}}\,\Delta\,(U_{\mathcal{B}}^{\mathsf{T}} x_{\mathcal{B}})$. Because the CLA accesses the covariance solely through the protocol, switching from a dense matrix to a factor model requires no change to the turning-point loop.

6 The efficient frontier object

The list of turning points is wrapped in a `Frontier` object (`src/cvxcla/types.py`). Each `FrontierPoint` carries a weight vector and exposes its expected return $\mu^{\mathsf{T}} w$ and variance $w^{\mathsf{T}}\Sigma w$. Between adjacent turning points the frontier in *weight* space is the straight segment (3), so the object can `interpolate` any number of intermediate portfolios by convex combination. Derived quantities — volatility, Sharpe ratio — are computed pointwise. The maximum-Sharpe portfolio is found by a one-dimensional search over the convex combination of the two turning points bracketing the discrete Sharpe maximum, exploiting the same piecewise-linear weight structure.

7 Complexity and properties

- **Exactness.** The frontier is returned exactly as a finite set of turning points; between them the closed-form segment (3) holds. No discretisation error is introduced.
- **Number of turning points.** Each iteration changes exactly one asset's status. Empirically the number of turning points is $O(n)$; the worst case is combinatorial (as for parametric active-set methods generally) but is not observed on the problems of §9. The implementation caps iterations at $100(n+1)$ as a numerical safety net.

- **Per-iteration cost.** Dominated by the free-block solve: $O(n_{\mathcal{F}}^3)$ for the dense backend, or $O(n_{\mathcal{F}}k^2 + k^3)$ for the factor backend via Woodbury (12). The Schur complement (5) adds only an $m \times m$ solve ($m = 1$ for the budget constraint).

- **Robustness on degenerate data.** The Bland-style tie-break (§4.4) and the $\sqrt{\varepsilon_{\text{mach}}}$ slope floor (§4.3) make the trace deterministic and prevent both cycling and out-of-bounds drift.

8 Relation to the Bailey–López de Prado algorithm

The most widely used open implementation of the CLA is that of Bailey and López de Prado (2013) (the algorithm behind PyPortfolioOpt's `CLA` (Martin, 2021), against which we benchmark in §9). It traces the same turning points as the method described here — the underlying mathematics is Markowitz's — but differs in four ways that matter in practice.

1. **Covariance representation.** Bailey–López de Prado operate on an *explicit dense covariance matrix*: each turning point forms and inverts the free-block covariance $\Sigma_{\mathcal{F}\mathcal{F}}$ directly (their `getMatrices` / `reduceMatrix` helpers slice the stored matrix). `cvxcla` never requires the matrix: it reaches the covariance only through the operator protocol of §5, so structured backends (the diagonal-plus-low-rank Woodbury solve) plug in unchanged. This is the central difference and the source of the asymptotic speed-up in §9.4.

2. **Linear algebra.** Where the reference algorithm forms an explicit inverse of the free block from scratch at each step (and, in the branch that decides which blocked asset re-enters, recomputes that full inverse once for every candidate), `cvxcla` solves the reduced KKT system by block elimination: a single multi-right-hand-side solve against $\Sigma_{\mathcal{F}\mathcal{F}}$ feeds an $m \times m$ Schur complement (5) that handles the equality constraints, and the event search over all candidates is vectorised rather than looped. The α and β systems reuse the same factorisation, and the covariance enters only as a black-box solve — which is what makes the backend abstraction possible.

3. **Constraint generality.** The reference implementation targets box bounds plus a single budget (fully-invested) constraint. `cvxcla` admits an arbitrary set of m linear equality constraints $Aw = b$; the budget is just the case $A = \mathbf{1}^\top$, $b = 1$, and the Schur complement scales to $m > 1$ at negligible cost.

4. **Degeneracy handling.** On tie-heavy or degenerate inputs several events can share the same critical λ. `cvxcla` applies an explicit Bland-style, lowest-index tie-break (§4.4) together with a floating-point slope floor (§4.3) to keep the trace deterministic and bounded; the reference implementation has no comparable anti-cycling rule. This hardens the common cases, though very degenerate problems remain delicate (§9).

These differences compound into a large runtime gap (§9). It would be tempting to dismiss the gap as "just an implementation detail," but the reference implementation is not slow for lack of `numpy` — it uses `numpy` for its per-step algebra. It is slow because of the structure above: a full free-block inverse rebuilt every step, and rebuilt once per candidate in the asset-entry search. §9.3 pins this down with a controlled baseline.

Relation to large-scale active-set methods. Bailey–López de Prado is the natural baseline because it is the most widely used open CLA, but it is not the only line of work on tracing the frontier efficiently. Perold (1984), Stein et al. (2008) and Hirschberger et al. (2010) develop parametric quadratic-programming and active-set methods aimed squarely at *large* universes, and they too drive the per-step cost well below a dense $O(n_{\mathcal{F}}^3)$ refactorisation. The decisive difference is *where* the speed-up comes from. Those methods accelerate the linear algebra of a covariance that is still treated as an explicit (if large and possibly sparse) matrix — through incremental factorisation updates, exploitation of sparsity, or specialised pivoting — so the gain is tied to a particular matrix representation baked into the solver. `cvxcla` instead leaves the turning-point loop entirely representation-agnostic: the covariance enters only through the three-operation operator protocol of §5, so the same loop runs unchanged on a dense matrix or on a diagonal-plus-low-rank factor model, and the asymptotic advantage of §9.4 is a property of the *backend*, not of the algorithm. The two ideas are complementary rather than competing: an incremental-update solver for the free block could itself be wrapped as a `CovarianceOperator` and dropped in. A direct empirical comparison against these methods — none of which ships a maintained Python implementation we are aware of — is left to future work; the reading here is that `cvxcla`'s contribution is the interface that makes such backends interchangeable, not a claim to the fastest dense large-scale solve.

9 Numerical experiment

To illustrate the algorithm we first trace the entire efficient frontier of a small problem — small enough that its piecewise-linear corner structure is plainly visible — and then measure how the runtime grows with the number of assets. Both experiments are reproducible from fixed random seeds: `experiments/frontier_20x50.py` and `experiments/runtime_scaling.py`.

9.1 Illustration: a 20-asset frontier

Setup. We simulate $T = 50$ daily returns for $N = 20$ assets from a $K = 5$ factor model. As the risk model we use the factor covariance itself, $\Sigma = \operatorname{diag}(d) + U\,\Delta\,U^{\mathsf{T}}$ (standard practice — a structured risk model rather than the noisy sample covariance), and we estimate the expected returns $\mu \in \mathbb{R}^{20}$ by the sample mean over the 50 days. We trace the long-only, fully-invested frontier, i.e. (1) with the budget constraint $\mathbf{1}^{\mathsf{T}} w = 1$ and box bounds $0 \le w \le 1$.

Results. The algorithm returns the *entire* frontier as a sequence of **21 turning points** (Figure 1); between consecutive points the frontier is the exact affine segment (3), so these 21 corner portfolios describe the whole efficient set with no discretisation. The piecewise-linear corners are clearly visible towards the low-variance end of the curve. Tracing this small problem takes on the order of a millisecond. Because the dense matrix and the structured `FactorCovariance` (Woodbury) backend represent the *same* Σ, the two backends return the identical 21-point frontier — the Woodbury identity is exact, so it changes only the cost of each solve, never the result; its runtime advantage as the universe grows is the subject of §9.2. (As an aside, replacing Σ by a low-rank *approximation* of the full sample covariance — its leading eigenpairs plus a diagonal residual — yields a near-identical frontier differing by at most one corner, which is why factor risk models are a practical substitute for the raw sample covariance.)

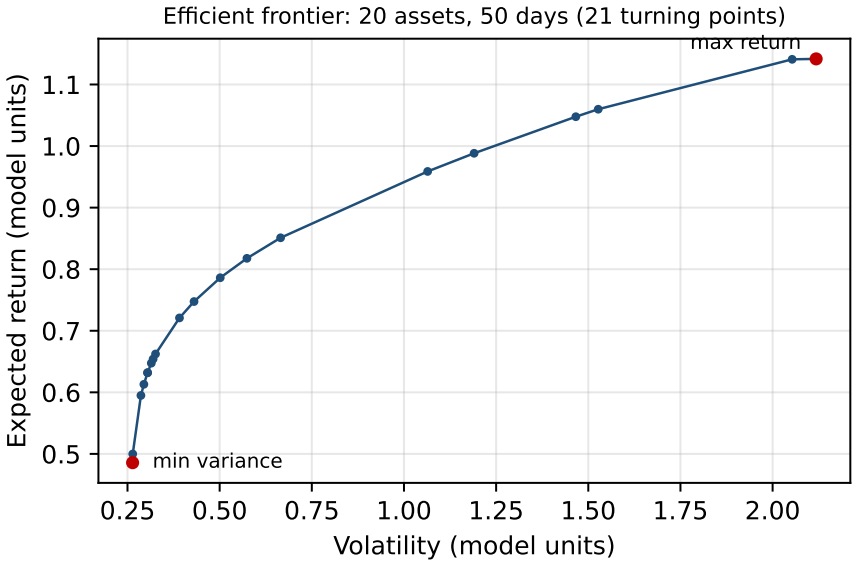

Figure 1: The efficient frontier of the 20-asset, 50-day problem in expected-return / volatility space, computed exactly by the Critical Line Algorithm. Each marker is one of the 21 turning points; the endpoints are the maximum-return portfolio ($\lambda = \infty$) and the global minimum-variance portfolio ($\lambda = 0$).

9.2 Runtime versus problem size

We next trace the full frontier for $n \in \{20, 40, 80, 160, 320, 640\}$ assets, each from a ground-truth $K = 10$ factor model so that the dense and factor backends again solve the identical Σ and return the same frontier. For reference we also time the `CLA` of PyPortfolioOpt (Martin, 2021), a widely-used implementation of the critical-line algorithm of Bailey and López de Prado (2013), on the same dense problem. The number of turning points tracks n closely ($21, 42, 81, 159, 322, 641$ respectively, and PyPortfolioOpt finds essentially the same set), confirming the $O(n)$ count of §9.

Figure 2 plots the median trace time against n on log–log axes. A least-squares fit over the largest four sizes gives an empirical exponent of ≈ 2.2 for the `cvxcla` dense backend and ≈ 1.3 for the factor backend: the dense per-iteration solve scales like $O(n_{\mathcal{F}}^3)$ over $O(n)$ iterations, while the Woodbury solve with fixed K stays close to linear in n. The dense and factor backends are comparable for small universes (the Woodbury correction carries a fixed $K \times K$ overhead), but the factor backend pulls away as n grows — from rough parity at $n = 40$ to a $\approx 8\times$ speedup at $n = 640$ (Table 1). This $\approx 8\times$ is the genuine *algorithmic* gain: both backends are vectorised `numpy` solving the identical Σ, so the only difference is the asymptotic Woodbury advantage of the structured solve over the dense one.

For external context we also time PyPortfolioOpt, which scales far more steeply ($\approx n^{3.4}$): at $n = 640$ it takes about 264 seconds, so `cvxcla`'s dense backend is roughly $340\times$ faster and the factor backend roughly $2900\times$ faster. This gap is not a `numpy`-versus-Python artefact — PyPortfolioOpt uses `numpy` for its linear algebra — but a consequence of how it organises the work: it rebuilds a full free-block inverse at every step, and once per candidate in the asset-entry search (§8). §9.3 separates this structural cost from the genuinely *algorithmic* advantage, which is the structured factor backend.

n	Turning points	PyPortfolioOpt [ms]	Inverse baseline [ms]	`cvxcla` dense [ms]	`cvxcla` factor [ms]
20	21	7.4	0.8	1.2	1.5
40	42	42.0	2.0	3.2	3.3
80	81	211.1	5.0	7.3	6.5
160	159	1,473	12.7	23.5	14.2
320	322	16,712	49.6	112.2	33.5
640	641	263,782	325.6	774.9	92.0

Table 1: Frontier trace time versus number of assets n ($K = 10$ factors). All four implementations are timed with the *same* protocol — the median of 3 repetitions on the same machine and inputs — and return the same $O(n)$-point frontier. "Inverse baseline" is the incremental-inverse CLA of §9.3 (`experiments/inverse_cla.py`): it shares `cvxcla`'s vectorised event logic but maintains $\Sigma_{\mathcal{FF}}^{-1}$ through rank-one updates instead of re-solving each step, so the gap between it and the dense backend isolates the linear-algebra strategy. Empirically the times scale like $\approx n^{3.4}$ (PyPortfolioOpt), $\approx n^{2.0}$ (inverse baseline), $\approx n^{2.2}$ (`cvxcla` dense) and $\approx n^{1.3}$ (`cvxcla` factor).

9.3 What the runtime gap is, and is not

The $\approx 340\times$ gap between `cvxcla`'s dense backend and PyPortfolioOpt at $n = 640$ invites a lazy explanation — "vectorised `numpy` versus slow Python" — that is wrong. PyPortfolioOpt already uses `numpy` (`np.linalg.inv`, `np.dot`) for its per-step linear algebra. It is slow because of *how* it organises the work: it rebuilds a full inverse of the free block from scratch at every step, and in the branch that decides which blocked asset should re-enter it recomputes that full $O(n_{\mathcal{F}}^3)$ inverse once for *every* candidate blocked asset — of order n dense inverses per turning point. `cvxcla` instead forms the affine segment once per step from a single solve and evaluates all event candidates simultaneously (the L matrix of §4.3), so the gap is structural and algorithmic, not a `numpy` artefact.

That still leaves a fair question about `cvxcla`'s own dense backend: it too re-solves from scratch each step, discarding the previous factorisation. Could a CLA that instead *maintains* the free-block inverse and updates it by rank-one formulas do better? We test exactly this with a third implementation (`experiments/inverse_cla.py`): a CLA that shares `cvxcla`'s vectorised event logic but maintains $\Sigma_{\mathcal{FF}}^{-1}$ through rank-one bordered (asset enters) and deletion (asset leaves) updates, $O(n_{\mathcal{F}}^2)$ per step against the fresh solve's $O(n_{\mathcal{F}}^3)$. It is *not* a reimplementation of PyPortfolioOpt — it avoids PyPortfolioOpt's per-candidate rebuild — and it traces the identical turning points to machine precision (max. weight discrepancy $< 10^{-13}$ across $n = 20$–160), so the comparison against the dense backend is clean: only the linear-algebra strategy differs.

The answer is yes: the incremental-inverse baseline ($\approx n^{2.0}$, 326 ms at $n = 640$) is about $2.4\times$ *faster* than the dense backend ($\approx n^{2.2}$, 775 ms), exactly as the $O(n_{\mathcal{F}}^2)$-versus-$O(n_{\mathcal{F}}^3)$ count predicts. `cvxcla`'s block elimination is therefore not the fastest possible dense strategy — and the package ships the incremental inverse as the opt-in `IncrementalDenseCovariance` backend (§5.2) for callers who want that win. It is not the *default* because the fresh solve is what composes cleanly with the covariance operator (§5) and is numerically robust step to step: the factor backend never forms $\Sigma_{\mathcal{FF}}$ at all, so there is no inverse to maintain — it solves through the Woodbury identity. The default dense backend gives up the modest incremental-inverse win to keep one turning-point loop that runs over *any* backend, and the factor backend more than repays it: at $n = 640$ it is $\approx 8.4\times$ faster than the dense backend and $\approx 3.5\times$ faster than the incremental-inverse baseline, the only one of the four implementations with a genuinely sub-quadratic exponent ($\approx n^{1.3}$). The structured solve, not the linear-algebra bookkeeping, is the real algorithmic advance.

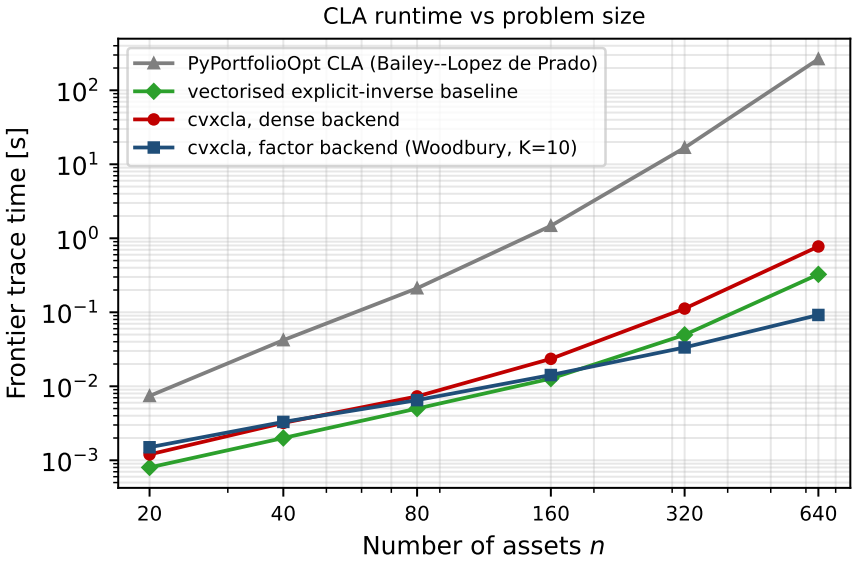

Figure 2: Frontier trace time as a function of the number of assets n (log–log). PyPortfolioOpt (Martin, 2021), a critical-line implementation in pure Python, scales like $\approx n^{3.4}$; the vectorised explicit-inverse baseline of §9.3 like $\approx n^{2.0}$; `cvxcla`'s dense backend like $\approx n^{2.2}$; and its diagonal-plus-low-rank factor backend, solving the identical covariance through the Woodbury identity, like $\approx n^{1.3}$. At $n = 640$ the factor backend is roughly 2900× faster than PyPortfolioOpt and 8× faster than the dense backend. The incremental-inverse baseline and the dense backend lie close together, far below PyPortfolioOpt.

9.4 Runtime versus factor rank

The advantage of the factor backend is not unconditional: it comes from the low-rank structure, so it must fade as the rank K approaches n. To make this precise we fix the universe at $n = 320$ assets and vary the number of factors $K \in \{20, 40, 80, 160, 320\}$ of the ground-truth covariance, tracing the same frontier with both backends. The dense backend factorises the same 320×320 free block regardless of K, so its time is essentially flat; the Woodbury solve costs $O(n_{\mathcal{F}} K^2 + K^3)$ per step and grows with K. Figure 3 and Table 2 show the crossover: the factor backend is 3.2× faster at $K = 20$, reaches parity near $K = 160$, and is about 2× *slower* at $K = 320$, where the "low-rank" part is full rank and the Woodbury correction is pure overhead. The practical reading is that the structured backend pays off precisely in the regime real risk models inhabit — a few tens of factors over hundreds or thousands of assets ($K \ll n$).

9.5 Exactness against a reference QP solver

The frontier is claimed to be *exact*: each segment (3) is the true optimiser of (1), not an interpolation. We verify this independently. Taking 40 real assets (§9.6, full-history sample covariance, condition number ≈ 200), we trace the frontier with the CLA (29 turning points) and then, at the midpoint λ of every one of the 27 interior segments, re-solve the parametric quadratic program (1) from scratch with a general convex solver (CVXPY/Clarabel (Diamond and Boyd, 2016), tight tolerances) and compare its optimiser against the CLA weights read off the segment by linear interpolation in λ. The agreement is at solver precision — median weight discrepancy 5×10^{-8}, maximum 1.3×10^{-5} —

K	`cvxcla` dense [ms]	`cvxcla` factor [ms]	Speedup
20	109.1	33.8	3.2×
40	110.9	42.6	2.6×
80	106.2	58.0	1.8×
160	86.5	83.2	1.0×
320	86.1	194.3	0.4×

Table 2: Frontier trace time at fixed $n = 320$ as the factor rank K varies (median of 3 repetitions). The dense backend is independent of K; the Woodbury backend grows with K and loses its advantage once K is no longer small relative to n. Both return the same ≈ 320-point frontier.

confirming that the affine segments coincide with the QP optima rather than approximating them (`experiments/validate_exact.py`).

9.6 Empirical example: the S&P 500 frontier

To close, we run the algorithm on real data. Using `experiments/fetch_sp500.py` we pull the current S&P 500 constituents from Wikipedia and download five years of daily adjusted-close prices via `yfinance`, yielding $N = 494$ assets over $T = 1213$ trading days (2021-07-30 to 2026-05-29) after dropping thinly-traded names. From the full-history sample mean and covariance (condition number $\approx 1.1 \times 10^5$) the CLA traces the entire long-only frontier — **96 turning points** — in a median of ≈ 14 ms (Figure 4); a rank-20 diagonal-plus-low-rank approximation of the same covariance traces a near-identical frontier (89 turning points) in ≈ 9 ms through the Woodbury backend. The annualised frontier spans expected returns of 0.14–0.89 over volatilities of 0.10–0.87, with a maximum Sharpe ratio of about 2.7. This figure is measured in-sample — the frontier is optimised over the same sample mean and covariance from which it is computed — and so overstates what is achievable out-of-sample; it is reported only to characterise the geometry of the frontier on a realistic problem, not as a claim of attainable performance.

Remark on conditioning and a real degeneracy. The traces above run cleanly because the expected returns are well separated and the covariance is well conditioned. The limitation is real, though, and the same data exhibits it: estimating the covariance from a short 120-day window leaves it rank-deficient ($\text{rank} = 119 < N = 494$), and the CLA then aborts with a bound violation (`Weights below lower bounds`) rather than completing the trace. This is the degeneracy weakness of the event logic — and it is not merely a conditioning artifact: linear shrinkage toward a scaled identity, which restores positive-definiteness, does not rescue the trace, so the issue is the coincident-event handling on large, near-degenerate problems. The Bland-style rule of §4.4 hardens the common cases and the $\sqrt{\varepsilon_{\text{mach}}}$ slope floor of §4.3 prevents out-of-bounds drift, but very tie-heavy or rank-deficient inputs remain a known limitation; a well-conditioned, full-rank estimate (ample history, or a structured factor model) is the reliable input.

10 Conclusion

The Critical Line Algorithm computes the complete mean–variance efficient frontier by recognising its piecewise-linear structure in weight space and walking from the maximum-return portfolio to the minimum-variance portfolio, flipping one asset between the free and blocked sets at each turning point. The `cvxcla` implementation realises each step as a single block-eliminated KKT solve,

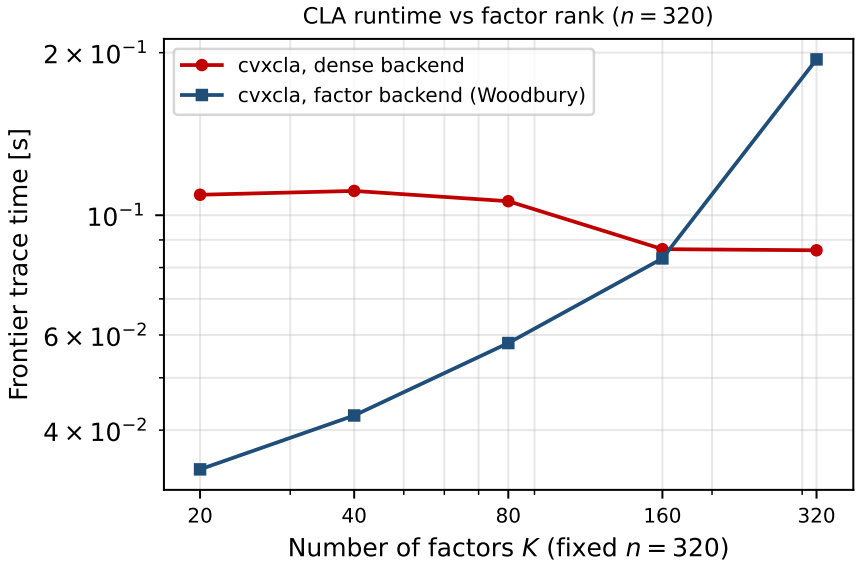

Figure 3: Frontier trace time at fixed $n = 320$ as a function of the factor rank K (log–log). The dense backend is flat in K; the diagonal-plus-low-rank factor backend grows like the Woodbury cost $O(n_{\mathcal{F}}K^2 + K^3)$ and crosses the dense line near $K \approx 160$. The structured backend wins only while $K \ll n$.

hardens the event logic against degeneracy and floating-point noise, and abstracts the covariance behind a small operator protocol so that structured risk models obtain the same exact frontier at a fraction of the dense cost.

Acknowledgements

The author thanks Stephen Boyd and Philipp Schiele. The first versions of the solver were implemented while the author stayed for a sabbatical at Boyd's group at Stanford University.

References

H. M. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

H. M. Markowitz. The optimization of a quadratic function subject to linear constraints. *Naval Research Logistics Quarterly*, 3(1–2):111–133, 1956.

H. M. Markowitz. *Portfolio Selection: Efficient Diversification of Investments*. John Wiley & Sons, New York, 1959.

A. Niedermayer and D. Niedermayer. Applying Markowitz's critical line algorithm. In *Handbook of Portfolio Construction*, pages 383–400. Springer, 2010.

D. H. Bailey and M. López de Prado. An open-source implementation of the critical-line algorithm for portfolio optimization. *Algorithms*, 6(1):169–196, 2013.

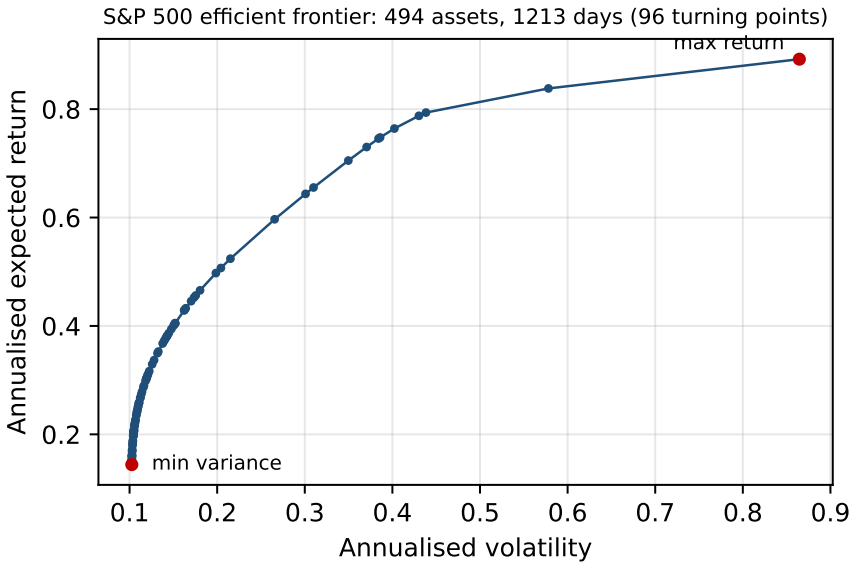

Figure 4: The long-only efficient frontier of 494 S&P 500 constituents from a full-history daily sample covariance ($T = 1213$ days), computed exactly by the CLA. Moments are annualised; each marker is one of the 96 turning points.

R. A. Martin. PyPortfolioOpt: portfolio optimization in Python. *Journal of Open Source Software*, 6(61):3066, 2021. https://doi.org/10.21105/joss.03066.

S. Diamond and S. Boyd. CVXPY: a Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

P. Wolfe. The simplex method for quadratic programming. *Econometrica*, 27(3):382–398, 1959.

W. F. Sharpe. A simplified model for portfolio analysis. *Management Science*, 9(2):277–293, 1963.

A. F. Perold. Large-scale portfolio optimization. *Management Science*, 30(10):1143–1160, 1984.

H. M. Markowitz and G. P. Todd. *Mean-Variance Analysis in Portfolio Choice and Capital Markets*. Frank J. Fabozzi Associates, New Hope, PA, 2000.

M. Stein, J. Branke, and H. Schmeck. Efficient implementation of an active set algorithm for large-scale portfolio selection. *Computers & Operations Research*, 35(12):3945–3961, 2008.

M. Hirschberger, Y. Qi, and R. E. Steuer. Large-scale MV efficient frontier computation via a procedure of parametric quadratic programming. *European Journal of Operational Research*, 204(3):581–588, 2010.